

Deep Learning for teens

By Hamid Sadeghi

Table of contents

- ▶ Review of last session
- ▶ Bidirectional RNN
- ▶ Encoder-Decoder RNN
- ▶ Bidirectional RNN vs Encoder-Decoder RNN
- ▶ stacked RNN
- ▶ Homework

Review of previous session

- ▶ What is RNN
- ▶ RNN architecture
- ▶ RNN Input-Output Types
- ▶ Vanishing memory in RNN
- ▶ LSTM
- ▶ GRU

All RNN Types

- ▶ **Vanilla RNN** → basic, simple memory
- ▶ **LSTM** → long-term memory, avoids vanishing gradient
- ▶ **GRU** → simpler than LSTM, similar performance
- ▶ **Bidirectional RNN** → reads sequence forward + backward
- ▶ **Encoder-Decoder RNN** → used for translation, seq2seq
- ▶ **Stacked RNN** → multiple RNN layers on top of each other

Vanilla RNN

- ▶ A Vanilla RNN (Simple RNN) is the most basic recurrent neural network.
- ▶ Its job is to read a sequence one step at a time and keep a running memory of what it has seen.

$$h_t = \tanh(Wx_t + Uh_{t-1} + b)$$

- **W** → transforms the current input
- **U** → transforms the previous memory
- **b** → bias
- **tanh** → activation function
- h_t → new hidden state (the memory)

```
x1 → [ RNN Cell ] → h1  
x2 → [ RNN Cell ] → h2  
x3 → [ RNN Cell ] → h3
```

Vanishing memory

$$h_t = U^t h_0 + \dots$$

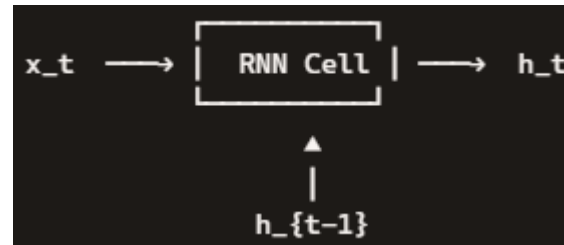
If the eigenvalues of **U** are less than 1 (which is common):

- U^2 becomes tiny
- U^5 becomes microscopic
- U^{20} becomes basically **zero**

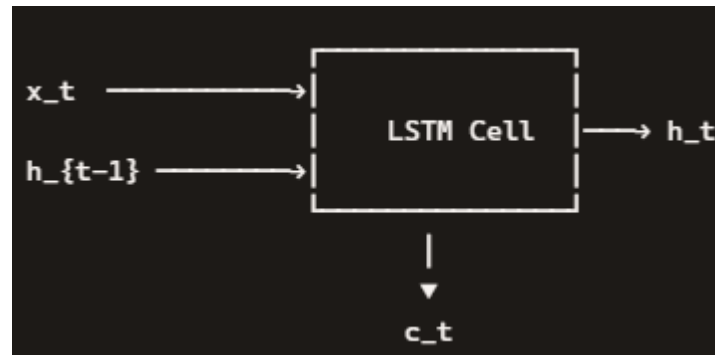
So the memory of early inputs disappears.

Simple Visualization

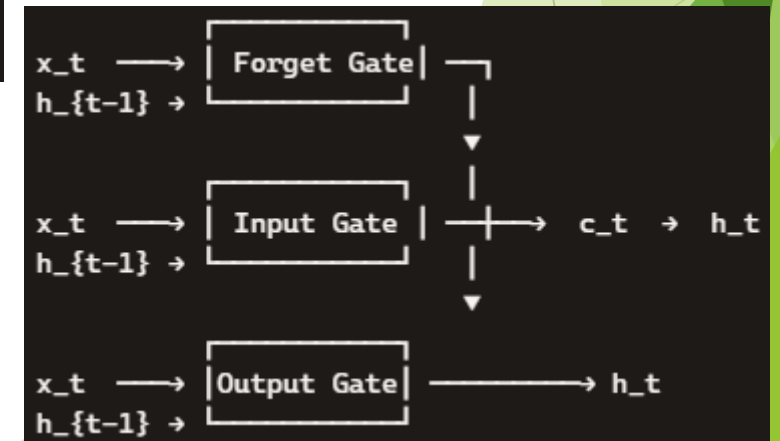
- ▶ Vanilla RNN – Simple Visualization



- ▶ LSTM – Simple Visualization



- ▶ Inside the LSTM cell:



LSTM

- ▶ Three gates
 - ▶ Forget
 - ▶ Input
 - ▶ Output
- ▶ Two states
 - ▶ Hidden state h_t
 - ▶ Short term memory
 - ▶ Cell state c_t
 - ▶ Long term memory

GRU

- ▶ LSTM, but lighter

Model	States	Gates	Complexity
Vanilla RNN	1 state (h)	0 gates	simple
LSTM	2 states (h, c)	3 gates	heavy
GRU	1 state (h)	2 gates	medium

- ▶ 1 state
- ▶ 2 gates
 - ▶ Update
 - ▶ Reset

Bidirectional RNN

- ▶ A regular RNN only remembers what came before, while a bidirectional RNN looks both ways—backward and forward—to get the full picture.
- ▶ Examples:
 - ▶ Predicting tomorrow from the last 12 days
 - ▶ Classifying a sequence
 - ▶ Sentiment analysis
 - ▶ Named entity recognition
 - ▶ Time-series regression on a fixed window
- ▶ Think of it like this:
 - ▶ “I want to understand the input better by reading it from both sides.”

Encoder-Decoder RNN

- ▶ A model that **reads an input sequence with an encoder and generates a new sequence with a decoder**, allowing input and output lengths to be different.
- ▶ The encoder reads the whole input and turns it into one meaning.
- ▶ The decoder takes that meaning and produces a new sequence from it.
- ▶ Or
 - ▶ Encoder → reads the 12-day sequence and compresses it into a context vector
 - ▶ Decoder → takes that context and predicts the next value

Encoder-Decoder RNN

- ▶ Encoder-Decoder is used when:
 - ▶ Input length $\neq$ Output length
- ▶ Examples:
 - ▶ Machine translation (English $\rightarrow$ French)
 - ▶ Summarization
 - ▶ Multi-step forecasting (12 days $\rightarrow$ next 7 days)
 - ▶ Any seq2seq task

Bidirectional RNN vs Encoder-Decoder RNN

- Bidirectional RNN: reads the same sequence in both directions to understand it better.
- Encoder-Decoder RNN: reads one sequence and generates a new sequence.

Model	Input	Output	Purpose
Bidirectional RNN	One sequence	One prediction or same-length sequence	Reads the input forward + backward
Encoder-Decoder RNN	One sequence	A new sequence (any length)	Transforms one sequence into another

stacked RNN

- ▶ A stacked RNN is simply multiple RNN layers placed on top of each other, so the output of one RNN becomes the input of the next.
- ▶ It lets the model learn deeper temporal patterns than a single-layer RNN ever could.

Assignment

- ▶ Please use the Sunspots dataset and evaluate all major RNN architectures to identify which model achieves the highest accuracy.